

Resumen Técnico del Firmware

Sistema de Control de Acceso – FRDM-K64F

Santiago Riverós Ignacio Sammartino Rodrigo Devesa Leandro Yiu

TP1 – Sistemas Embebidos

Resumen

Este documento resume, con fragmentos de código reales del firmware, las decisiones de diseño más importantes del sistema: la separación en capas APP/HAL/MCAL, la máquina de estados principal, el driver genérico de display, el manejo de interrupciones y el driver del lector de banda magnética. No es una descripción exhaustiva del código: el objetivo es explicar el *por qué* detrás de las partes menos obvias de la implementación.

Índice

1. Arquitectura en capas: APP → HAL → MCAL	1
2. Configuración centralizada y portabilidad	2
3. Máquina de estados principal	3
4. Agrupación de estado relacionado en <i>structs</i>	4
5. Temporización no bloqueante	5
6. Ventana deslizante para buffers más largos que el display	5
7. Secciones críticas en el driver de display	6
8. Despacho genérico de interrupciones de GPIO	7
9. Driver del lector de banda magnética	8
10. Prioridades de interrupción ajustadas por timing	9

1. Arquitectura en capas: APP → HAL → MCAL

El firmware está organizado en tres capas con una regla estricta de dependencia: la capa de **aplicación** (APP_layer) sólo puede llamar a la capa **HAL** (HAL_layer), y ésta a su vez sólo puede llamar al **MCAL** (MCAL_layer, acceso directo a los periféricos). La única excepción documentada es que el MCAL puede invocar, a través de un puntero a función, un callback registrado por la capa HAL (por ejemplo, el tick del SysTick llamando a la tarea de refresco del display).

Un caso concreto de esta regla en acción: originalmente, `App.c` llamaba directamente a funciones del MCAL (`SysTick_Init`, `NVIC_SetPriority`), lo que violaba la capa HAL. La solución fue introducir una fachada delgada, `scheduler.h/.c`, que no agrega lógica propia: sólo reenvía cada llamada al MCAL correspondiente, de modo que la capa APP nunca necesita incluir un header del MCAL.

```

1 void HAL_Scheduler_Init(void)
2 {
3     (void)SysTick_Init();
4 }
5
6 bool HAL_Scheduler_AddTask(hal_task_t tarea)
7 {
8     /* systick_callback_t y hal_task_t tienen la misma firma: la
9        fachada
10        * no le agrega semantica, solo evita que la APP conozca al MCAL.
11        */
12     return SysTick_AddCallback((systick_callback_t)tarea);
13 }
14
15 void HAL_Scheduler_GlobalIRQDisable(void)
16 {
17     __disable_irq();
18 }
19
20 void HAL_Scheduler_GlobalIRQEnable(void)
21 {
22     __enable_irq();
23 }

```

Listing 1: Fachada HAL sobre el SysTick (scheduler.c): reenvío puro, sin lógica propia.

Gracias a esta fachada, App_Init() puede deshabilitar las interrupciones globales durante todo el arranque (mientras los periféricos todavía no están configurados) y rehabilitarlas recién al final, sin que la capa de aplicación toque directamente el core:

```

1 void App_Init(void)
2 {
3     HAL_Scheduler_GlobalIRQDisable();
4
5     HAL_Scheduler_Init();
6     /* ... inicializacion de todos los modulos HAL ... */
7
8     HAL_Scheduler_GlobalIRQEnable();
9 }

```

Listing 2: Arranque seguro en App_Init(): interrupciones cortadas mientras el hardware se configura.

2. Configuración centralizada y portabilidad

Los valores que dependen del hardware o del ajuste fino de la aplicación viven en archivos _cfg.h dedicados (display_cfg.h, fsm_cfg.h), separados de la lógica que los usa. Esto permite portar el driver de display a otra placa, u otro proyecto, tocando *un solo archivo*.

El ejemplo más claro es la macro DISPLAY_COMPOSE, que define cómo se arma la trama binaria que se envía a los shift-registers 74HC595. El motor gráfico del driver (display.c) no sabe cuántos registros hay ni en qué byte cae cada campo: sólo calcula *qué* segmentos y selector hay que mostrar, y delega el *cómo empaquetarlos* en esta macro.

```

1 /* CABLEADO ACTUAL: dos registros; el primero en salir (U2) lleva los
2    bits de
3    * control (selector + LEDs) y el segundo (U1) los segmentos. */
4 #define DISPLAY_COMPOSE(frame, segs, sel, aux) \

```

```

4     do {
5         (frame)[0] = (uint8_t)((aux) | (sel));    /* control */ \
6         (frame)[1] = (uint8_t)(segs);           /* segmentos*/ \
7     } while (0)

```

Listing 3: DISPLAY_COMPOSE en display_cfg.h: el único lugar que conoce el cableado real de los shift-registers.

De la misma forma, fsm_cfg.h centraliza todos los tiempos y umbrales de la máquina de estados (duración de mensajes, cantidad de intentos fallidos, prioridad de interrupción del SysTick), con `_Static_assert` que verifican en tiempo de compilación que la configuración sea coherente:

```

1 #define FSM_MS_ACCESO_MENSAJE      2000U    /* "OPEn" fijo, para
   leerlo */
2 #define FSM_MS_ACCESO_TOTAL        5000U    /* total con el pestillo
   abierto */
3
4 _Static_assert(FSM_MS_ACCESO_MENSAJE <= FSM_MS_ACCESO_TOTAL,
5               "OPEn fijo no puede durar mas que el total de puerta
   abierta");

```

Listing 4: Chequeos de coherencia en tiempo de compilación (fsm_cfg.h).

3. Máquina de estados principal

El comportamiento de la aplicación es una FSM clásica de eventos y estados, declarada en fsm.h y despachada en FSM_Update(). Los eventos representan entradas del sistema (giros del encoder, lectura de tarjeta, timeout de inactividad); los estados representan las pantallas / modos en los que puede estar el sistema.

```

1 typedef enum
2 {
3     EV_NONE,
4     EV_ENCODER_GIRO_DER, EV_ENCODER_GIRO_IZQ,
5     EV_ENCODER_GIRO_PRESIONADO_DER, EV_ENCODER_GIRO_PRESIONADO_IZQ,
6     EV_ENCODER_PRESIONA, EV_ENCODER_DOBLE_TOQUE,
7     EV_ENCODER_MANTENER, EV_ENCODER_LIBERA_PERILLA,
8     EV_CARD_OK, EV_CARD_ERROR,
9     EV_TIMEOUT
10 } fsm_event_t;
11
12 typedef enum
13 {
14     STATE_STANDBY, STATE_LLECTURA_ID, STATE_INGRESO_PIN,
15     STATE_ACCESO_OK, STATE_PIN_INCORRECTA_DELAY, STATE_BLOQUEO_ID,
16     STATE_MODO_ADMIN, STATE_ERROR
17 } fsm_state_t;

```

Listing 5: Enumeraciones de eventos y estados de la FSM (fsm.h).

El despacho es un `switch` sobre el estado actual: cada estado es una función `SubFSM_*` independiente que recibe el evento y devuelve el próximo estado. Antes de llegar a ese switch, *dos* eventos se interceptan de forma global porque aplican sin importar en qué estado esté el sistema: el reset por mantener presionada la perilla, y el ajuste de brillo.

```

1 void FSM_Update(fsm_event_t evento)
2 {
3     /* RESET GLOBAL: disponible desde cualquier estado, salvo
   penalizado */

```

```

4   if (evento == EV_ENCODER_MANTENER)
5   {
6       if (f_penalizado == false)
7       {
8           FSM_Init();
9           return;
10      }
11  }
12
13  /* AJUSTE DE BRILLO GLOBAL: idem, no pasa por ningun sub-estado
14     */
15  if (evento == EV_ENCODER_GIRO_PRESIONADO_DER)
16  {
17      HAL_Brightness_Display(INCREASE);
18      return;
19  }
20  /* ... */
21
22  switch (estado_actual)
23  {
24      case STATE_STANDBY:      estado_actual = SubFSM_StandBy(evento
25                              );      break;
26      case STATE_LECTURA_ID:  estado_actual = SubFSM_Lectura_ID(
27                              evento);  break;
28      /* ... resto de los estados ... */
29  }
30 }

```

Listing 6: FSM_Update(): eventos globales interceptados antes del despacho por estado.

4. Agrupación de estado relacionado en *structs*

Cada submáquina de la FSM que necesita más de una variable de contexto las agrupa en un **struct** propio, en vez de dejarlas como variables sueltas del archivo. Esto deja explícito qué datos pertenecen a qué submáquina, y evita que se puedan actualizar de forma inconsistente entre sí (por ejemplo, tocar el cursor del menú admin sin resetear también el flag de refresco).

```

1  typedef struct
2  {
3      admin_substate_t  substate;
4      uint8_t          cursor;      /* opcion resaltada del
5                                   menu      */
6      uint8_t          usuario_sel; /* indice del usuario en
7                                   gestion */
8      bool             f_creando_usuario; /* alta nueva vs edicion
9                                   */
10     bool             f_refrescar;      /* se acaba de (re)entrar
11                                   */
12 } admin_ctx_t;
13
14 static admin_ctx_t s_admin =
15 {
16     .substate = ADMIN_VIEW_USERS, .cursor = 0U, .usuario_sel = 0U,
17     .f_creando_usuario = false, .f_refrescar = true
18 };

```

Listing 7: Contexto del modo administrador agrupado en un *struct* (fsm.c).

De forma análoga, todos los estados con pantalla temporizada (acceso concedido, error, bloqueo, ID duplicado) comparten *una sola* instancia de un cronómetro genérico, porque nunca hay dos activos al mismo tiempo:

```

1 typedef struct
2 {
3     bool      f_activo;
4     uint32_t  inicio_ms;
5     uint8_t   fase;      /* 0 = mensaje fijo, 1 = animacion (segun el
6                          estado) */
7 } cronometro_t;

```

Listing 8: Cronómetro genérico reutilizado por todos los estados temporizados.

5. Temporización no bloqueante

Todo el firmware evita `delay()` bloqueantes: los estados temporizados comparan la marca de tiempo guardada contra `HAL_GetTick()` en cada vuelta del súper-loop. Esto permite que el sistema siga respondiendo a la interfaz (encoder, lector) mientras, por ejemplo, se muestra el mensaje de acceso concedido.

```

1 static fsm_state_t SubFSM_Acceso_OK(fsm_event_t evento)
2 {
3     uint32_t transcurrido;
4
5     if (s_cron.f_activo == false)
6     {
7         s_cron.f_activo = true;
8         s_cron.inicio_ms = HAL_GetTick();
9         HAL_Message_Disp("OPEn");
10        HAL_Buzzer_On();
11    }
12
13    transcurrido = HAL_GetTick() - s_cron.inicio_ms;
14
15    if (transcurrido >= FSM_MS_ACCESO_TOTAL)
16    {
17        s_cron.f_activo = false;
18        HAL_Buzzer_Off();
19        FSM_Init();
20        return STATE_STANDBY;
21    }
22    return STATE_ACCESO_OK;
23 }

```

Listing 9: Patrón de temporización no bloqueante en `SubFSM_Acceso_OK()`.

6. Ventana deslizante para buffers más largos que el display

El display físico tiene 4 dígitos, pero el ID a ingresar tiene 8 y el PIN hasta 5. La función `Mostrar_VentanaIngreso` resuelve esto dibujando un recorte de 4 posiciones del buffer lógico que se corre a medida que el cursor avanza, de forma que el dígito en edición quede *siempre* visible en la última posición física una vez que el cursor sale de las primeras 4.

```

1 static void Mostrar_VentanaIngreso(const uint8_t *buffer, uint8_t
    cursor, bool f_oculto)

```

```

2 {
3     uint8_t ancho  = HAL_Display_GetDigits();
4     uint8_t offset = (cursor >= ancho) ? (uint8_t)(cursor - ancho + 1
5         U) : 0U;
6     uint8_t p;
7
8     HAL_Blink_Digit_Dis(ALL, STOP);
9
10    for (p = 0U; p < ancho; p++)
11    {
12        uint8_t logico = (uint8_t)(p + offset);
13
14        if (logico < cursor)
15        {
16            /* Ya confirmado: oculto ('-') para PIN, valor real para
17               ID. */
18            HAL_Send_Digit_Dis(p, f_oculto ? (disp_char_t)'-'
19                : Glifo_FSM(buffer[logico
20                    ]));
21        }
22        else if (logico == cursor)
23        {
24            /* Cursor: siempre descubierto y titilando. */
25            HAL_Send_Digit_Dis(p, Glifo_FSM(buffer[logico]));
26            HAL_Blink_Digit_Dis(p, START);
27        }
28        else
29        {
30            HAL_Clear_Digit_Dis(p);
31        }
32    }
33 }

```

Listing 10: Ventana deslizante: proyecta un buffer lógico largo sobre 4 dígitos físicos.

7. Secciones críticas en el driver de display

El driver de display expone su API pública desde el contexto de la aplicación, pero el contenido de la pantalla también lo modifica `HAL_Refresh_Task()`, que corre dentro de la ISR del `SysTick`. Para que ninguna de las dos partes vea una VRAM a medio escribir, toda función pública protege su acceso con una sección crítica mínima basada en `PRIMASK`:

```

1 static inline uint32_t disp_lock(void)
2 {
3     uint32_t primask = __get_PRIMASK();
4     __disable_irq();
5     return primask;
6 }
7
8 static inline void disp_unlock(uint32_t primask)
9 {
10    __set_PRIMASK(primask);
11 }
12
13 void HAL_Clear_Digit_Dis(uint8_t target)
14 {
15     uint32_t key = disp_lock();

```

```

16  /* ... modifica vram[] y s_blink.mascara ... */
17  disp_unlock(key);
18  }

```

Listing 11: Sección crítica mínima usando PRIMASK (display.c).

Guardar y restaurar el PRIMASK (en vez de simplemente deshabilitar y rehabilitar las interrupciones) hace que estas secciones críticas sean anidables sin efectos secundarios: si una función protegida llama a otra función protegida, la interna no reabre las interrupciones antes de tiempo.

Por requisito de la cátedra, el pin PIN_TEST_ISR se pone en alto mientras corre la ISR de refresco, lo que permite medir con osciloscopio su tiempo de ejecución real:

```

1  void HAL_Refresh_Task(void)
2  {
3  #if (DISPLAY_USE_TEST_PIN != 0)
4      gpioWrite(PIN_TEST_ISR, HIGH);
5  #endif
6      /* ... bit-banging de la trama hacia los 74HC595 ... */
7  #if (DISPLAY_USE_TEST_PIN != 0)
8      gpioWrite(PIN_TEST_ISR, LOW);
9  #endif
10 }

```

Listing 12: Instrumentación para medir la duración de la ISR con osciloscopio.

8. Despacho genérico de interrupciones de GPIO

El MCAL implementa un mecanismo de callbacks por pin para las interrupciones de puerto, en vez de que cada handler conozca de antemano qué pines usa cada driver. Una tabla de 5×32 punteros a función (un puerto, 32 pines) permite que cualquier módulo (encoder, lector de tarjetas) se registre con gpioIRQ() sin que el MCAL sepa qué hace ese callback.

```

1  static pinIrqFun_t irq_callbacks[5][32] = {0};
2
3  static void gpioIRQDispatch(uint8_t portId)
4  {
5      uint32_t flags = port_ptrs[portId]->ISFR;
6      uint8_t i;
7
8      for (i = 0U; i < 32U; i++)
9      {
10         if ((flags & (1UL << i)) != 0UL)
11         {
12             port_ptrs[portId]->ISFR = (1UL << i); /* se limpia escribiendo 1 */
13
14             if (irq_callbacks[portId][i] != 0)
15             {
16                 irq_callbacks[portId][i]();
17             }
18         }
19     }
20 }
21
22 void PORTB_IRQHandler(void) { gpioIRQDispatch(PB); }

```

Listing 13: Tabla de callbacks y despacho genérico de IRQ de puerto (gpio.c).

Un detalle importante de la implementación es limpiar el flag de interrupción *antes* de invocar al callback: si se limpiara después, un flanco nuevo que llegue mientras corre el callback se perdería junto con el flag viejo.

9. Driver del lector de banda magnética

El driver del lector separa deliberadamente la parte *urgente* de la *costosa*: la ISR de CLOCK sólo guarda un bit por interrupción (mínima y determinística), mientras que la decodificación de la trama Track 2 completa (búsqueda de sentinelas, chequeo de paridad) se hace en el lazo principal, nunca dentro de una ISR.

```
1 void IRQ_CardClk()
2 {
3     bitarray[currbit++] = !gpioRead(PIN_CARD_DATA); /* DATA es activo
4         bajo */
5     if (currbit >= MAX_BIT_LENGTH)
6     {
7         status = LECTOR_TIMEOUT;
8         gpioIRQ(PIN_CARD_CLK, GPIO_IRQ_MODE_DISABLE, IRQ_CardClk);
9     }
10 }
```

Listing 14: ISR mínima: un bit por interrupción (`lector.c`).

Un problema real detectado en la placa: el pin ENABLE del lector puede rebotar mecánicamente al apoyar la tarjeta, generando un pulso cortísimo que la ISR interpreta como un swipe completo con 0–3 bits capturados. Sin validación, esos bits (basura de una lectura anterior) se interpretaban como una tarjeta real. La solución fue exigir una cantidad mínima de bits capturados antes de aceptar la lectura como válida:

```
1 #define MIN_VALID_BITS 30
2 /* Una trama valida de track 2 con el ID de 8 digitos ronda los 50+
3    bits;
4    * un rebote deja 0-3 bits. Margen generoso. */
5 void IRQ_CardEnable()
6 {
7     /* ... flanco de bajada: arranca el swipe ... */
8     else
9     {
10         /* Flanco de subida: termino el swipe. */
11         gpioIRQ(PIN_CARD_CLK, GPIO_IRQ_MODE_DISABLE, IRQ_CardClk);
12
13         if (currbit < MIN_VALID_BITS)
14         {
15             /* Rebote de ENABLE o ruido: se descarta aca mismo, sin
16                pasar
17                * por RECIEVED, para que nunca se vea como un intento de
18                * lectura fallido en la aplicacion. */
19             status = LECTOR_WAITING;
20             return;
21         }
22         status = LECTOR_RECIEVED;
23     }
24 }
```

Listing 15: Rechazo de rebotes de ENABLE por cantidad insuficiente de bits (`lector.c`).

10. Prioridades de interrupción ajustadas por timing

La ISR del SysTick (que hace el *bit-banging* de la trama del display) es la más larga del sistema. Si tuviera la misma prioridad que la IRQ del puerto del lector de tarjetas, podría retrasar la captura de un flanco de CLOCK durante el swipe y perder un bit. La solución fue bajarle la prioridad al SysTick para que la IRQ del lector siempre pueda interrumpirlo:

```
1  /* La ISR del SysTick (bit-banging del display + encoder) es la mas  
2  larga  
3  * del sistema y, con la misma prioridad que PORTB (0 por defecto  
4  ambas),  
5  * retrasaria la captura de cada bit del lector. Se le baja la  
6  prioridad  
7  * al SysTick para que los flancos de CLOCK la puedan interrumpir  
8  siempre. */  
9  HAL_Scheduler_SetPriority(APP_NVIC_PRIO_SYSTICK);    /*  
10 APP_NVIC_PRIO_SYSTICK = 1 */  
11 inicializarLector();                                /* PORTB queda en  
12 prioridad 0 (default) */
```

Listing 16: Ajuste de prioridad de interrupción documentado en App_Init().

Esta decisión se verificó de forma empírica con osciloscopio, comparando el pin de instrumentación de la ISR (PIN_TEST_ISR) contra el pin ENABLE del lector durante un swipe real, confirmando que ningún pulso de la ISR del SysTick bloquea una ventana donde debería llegar un bit del lector.